



CampusFix

Software Requirements Specification (SRS)

Course: Software Engineering

Instructor: Sir Jahanzaib Ahmed Khan

Group Members:

Muhammad Azam (IU05-0124-0190)

Kashif Raza (IU05-0124-0303)

Tehreem Aijaz (IU05-0124-0126)

1. Introduction

1.1 Purpose

This Software Requirements Specification describes the requirements, assumptions, users, features, interfaces, constraints and acceptance criteria for CampusFix. It is written for the course instructor, project team and future developers who may review or extend the system.

1.2 Document Conventions

Functional requirements are labeled as FR-01, FR-02 and so on. Non-functional requirements are labeled as NFR-01 and onwards. The word shall indicates a required system behavior.

1.3 Intended Audience

- Course instructor and evaluator
- Project team members
- Students presenting the system in viva
- Future developer maintaining the project

1.4 Product Scope

CampusFix is a local desktop system for submitting, tracking and managing campus maintenance complaints. It is meant for small campus-level use and academic demonstration.

1.5 References

Python official documentation, Tkinter documentation, SQLite documentation and standard IEEE-style SRS structure were used as general references while preparing this document.

2. Overall Description

2.1 Product Perspective

CampusFix is a stand-alone desktop application. It does not depend on a web server. It stores data in a local SQLite database file and uses a Tkinter graphical user interface.

2.2 Product Functions

- User registration and login
- Role-based dashboard display
- Complaint creation with category, location and priority
- User complaint tracking
- Admin complaint filtering and search
- Complaint assignment and status update
- Dashboard counts for total, open, resolved and urgent complaints

2.3 User Classes and Characteristics

User Class	Description	Skill Level
Student/Staff User	Submits and tracks complaints	Basic computer usage
Admin/Maintenance User	Reviews complaints and updates progress	Basic to moderate computer usage

2.4 Operating Environment

Item	Requirement
Operating System	Windows, Linux or macOS with Python installed
Runtime	Python 3.x
Database	SQLite local database file
Display	Desktop or laptop screen suitable for Tkinter GUI

2.5 Design and Implementation Constraints

- The system uses Tkinter to avoid external GUI dependencies.
- The database is local SQLite, so multi-device simultaneous access is outside scope.
- The application is designed for a semester project and not for production deployment.

2.6 Assumptions and Dependencies

- Users will run the application on a computer with Python installed.
- The admin account will be available for demonstration.
- The local database file will remain in the project src folder unless the configuration is changed.

3. SDLC Model

The project uses the Iterative and Incremental Model. The reason for using this model is that CampusFix can be divided into small working increments: authentication, user complaint entry, admin complaint management, dashboard statistics and testing. After each increment, the team can run the application, check the screen behavior and improve the next part. This model is suitable because it reduces risk and gives visible progress during a semester project.

4. System Features and Functional Requirements

ID	Requirement	Detailed Description
FR-01	User Registration	The system shall allow new student/staff users to create an account with name, email and password.
FR-02	Login	The system shall authenticate users and open the correct dashboard based on role.
FR-03	Submit Complaint	A user shall submit a complaint with title, category, location, priority and description.
FR-04	Track Complaints	A user shall view his or her complaint history and current status.
FR-05	Admin View	An admin shall view all submitted complaints in one dashboard.
FR-06	Search and Filter	An admin shall filter complaints by status and search by title, reporter, category or complaint ID.
FR-07	Assign Complaint	An admin shall assign a complaint to a staff member or technician.
FR-08	Update Status	An admin shall update the status as Submitted, Assigned, In Progress, Resolved or Closed.
FR-09	Dashboard Counts	The system shall display total, open, resolved and urgent complaint counts.

5. External Interface Requirements

5.1 User Interface

The GUI shall provide separate views for login/registration, user dashboard and admin dashboard. Tables shall be used to display complaint records. Input fields shall be clearly labeled.

5.2 Software Interfaces

The application shall interact with SQLite through the database module. The GUI module shall call database functions instead of directly writing SQL inside button events where possible.

5.3 Hardware Interfaces

No special hardware is required. A standard desktop or laptop computer is sufficient.

5.4 Communication Interfaces

No internet or network communication is required in the current version.

6. Non-Functional Requirements

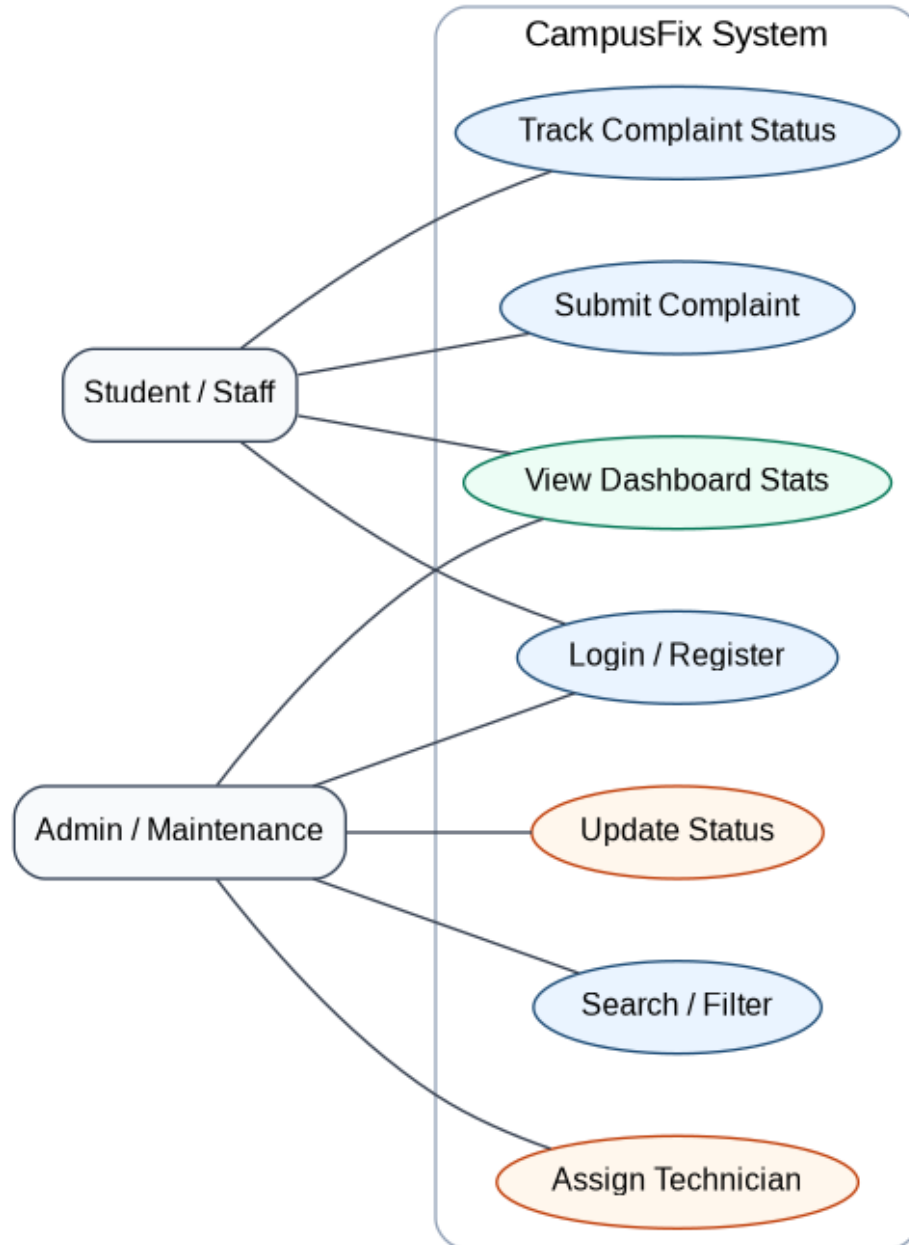
ID	Quality Attribute	Requirement
NFR-01	Usability	The interface should be simple enough for a first-time user to submit a complaint without training.
NFR-02	Reliability	SQLite foreign keys and required fields should protect the basic consistency of complaint records.
NFR-03	Performance	Common dashboard operations should respond quickly for a small campus dataset.
NFR-04	Maintainability	The code should keep GUI, configuration and database functions separate.
NFR-05	Security	Passwords are not stored directly; the project stores SHA-256 password hashes for demonstration.

7. Data Requirements

Entity	Attributes	Notes
User	id, full_name, email, password_hash, role, created_at	Email must be unique. Role controls the dashboard.
Complaint	id, user_id, title, category, location, priority, description, status, assigned_to, admin_note, created_at, updated_at	Complaint belongs to one user and can be updated by admin.

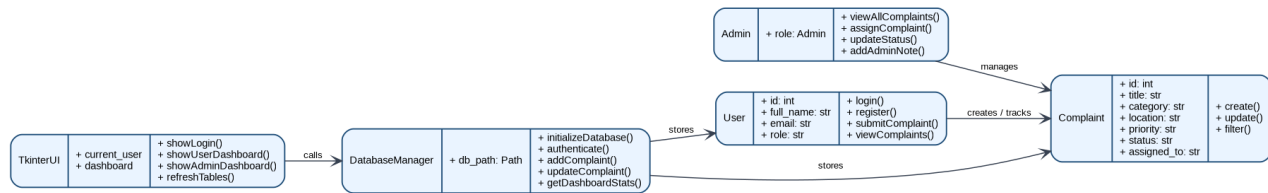
8. Diagrams

8.1 Use Case Diagram



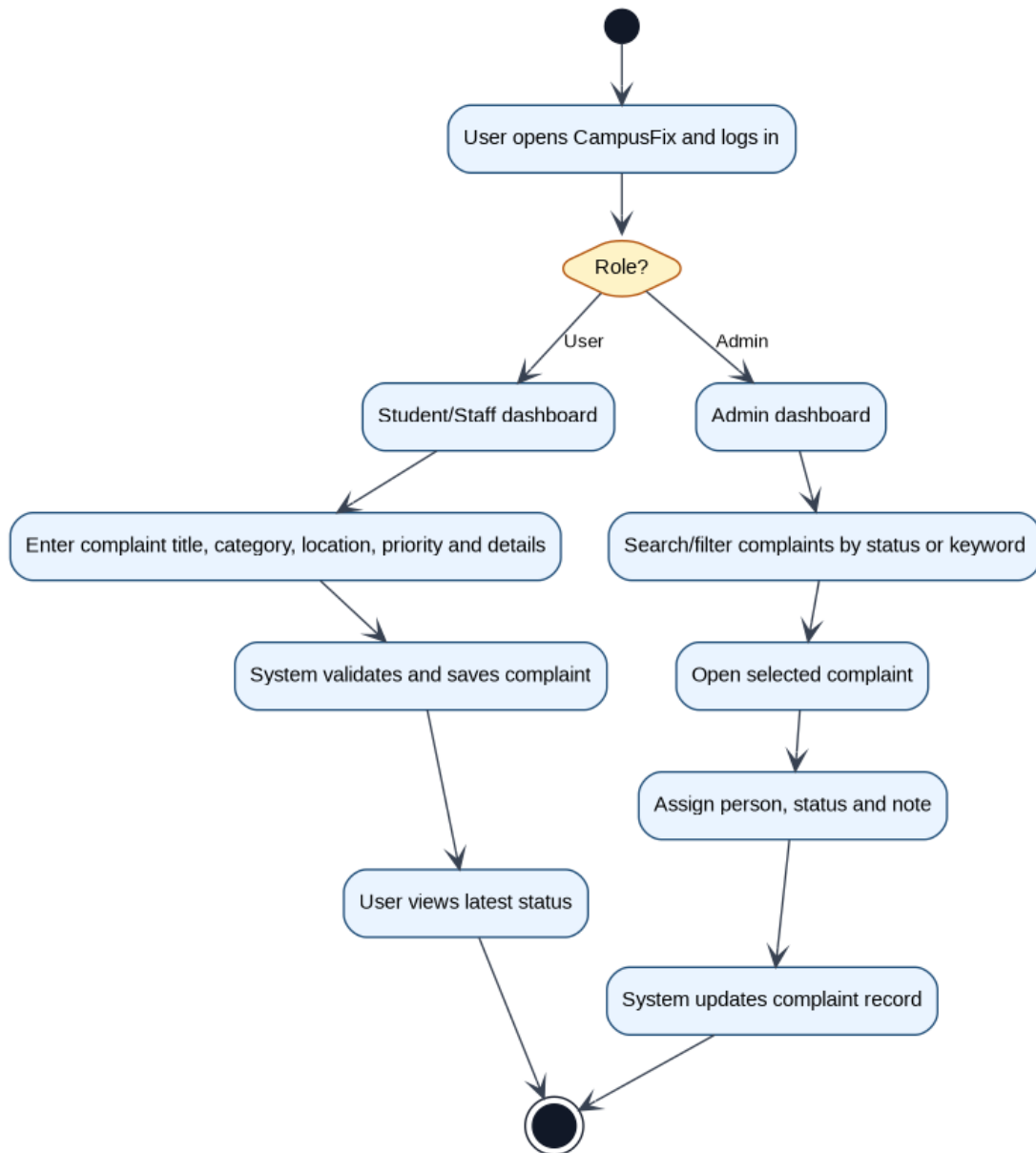
Use case diagram for CampusFix.

8.2 Class Diagram



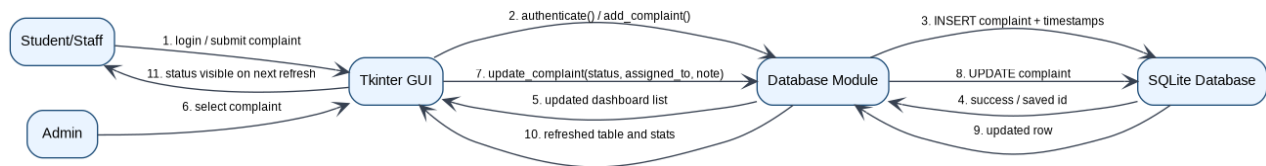
Class/module diagram for CampusFix.

8.3 Activity Diagram



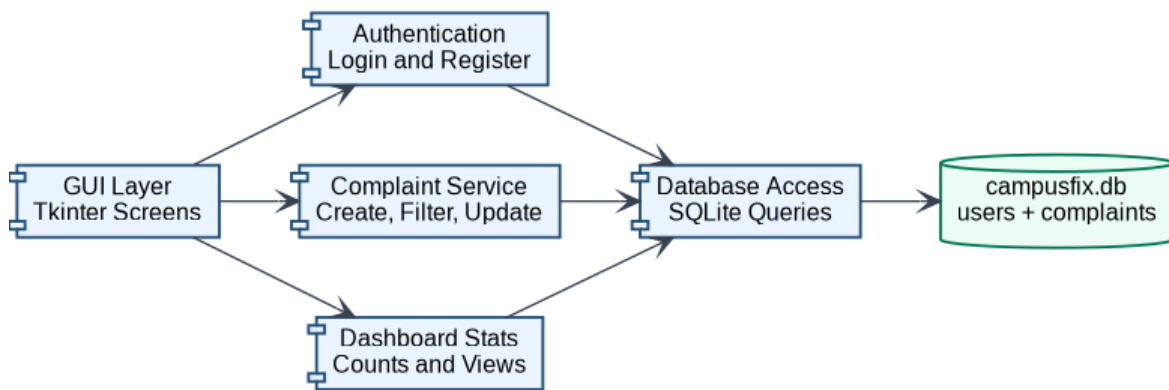
Activity diagram for complaint submission and management.

8.4 Sequence Diagram



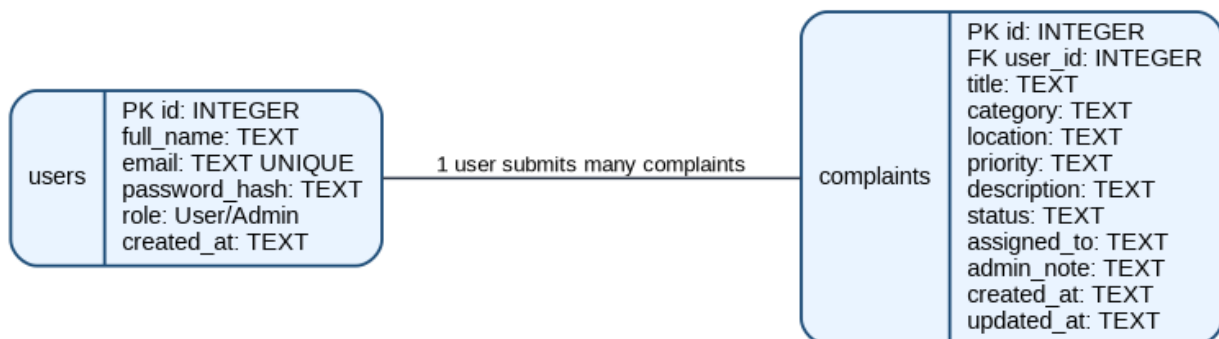
Sequence diagram for complaint life cycle.

8.5 UML Component Diagram



UML component diagram of the system layers.

8.6 Entity Relationship Diagram



ERD showing users and complaints tables.

9. Use Case Specifications

Use Case	Primary Actor	Precondition	Main Flow	Postcondition
UC-01 Register Account	Student/Staff	User is not registered	Enter name, email and password, then submit form	New user account is saved
UC-02 Submit Complaint	Student/Staff	User is logged in	Fill complaint form and press submit	Complaint is saved with Submitted status
UC-03 Track Complaint	Student/Staff	User has complaints	Open dashboard complaint table	User sees current status
UC-04 Manage Complaint	Admin	Admin is logged in	Search/select complaint, set status, assign staff and save note	Complaint record is updated

10. Acceptance Criteria

- A new user can register and log in successfully.
- A user can submit a complaint and see it in the complaint table.
- An admin can view all complaints and filter by status.
- An admin can update the status, assignment and notes of a complaint.
- The dashboard statistics change according to database records.
- The application can be run from the included project files without a complex installation process.

11. Requirement Traceability Matrix

Requirement	Design Element	Test Case
FR-01	Registration form and users table	TC-02
FR-02	Login screen and authenticate function	TC-01
FR-03	Complaint form and complaints table	TC-03
FR-06	Admin table search/filter controls	TC-04
FR-07/FR-08	Admin update panel and update complaint function	TC-05
FR-09	Dashboard stats function	TC-06

12. Future Requirements

Future versions may include web access, email notifications, image attachments, technician accounts, complaint SLA reports and role management. These features are intentionally not included in the first version to keep the project focused and presentable.